30 years

# Computer Science & IT

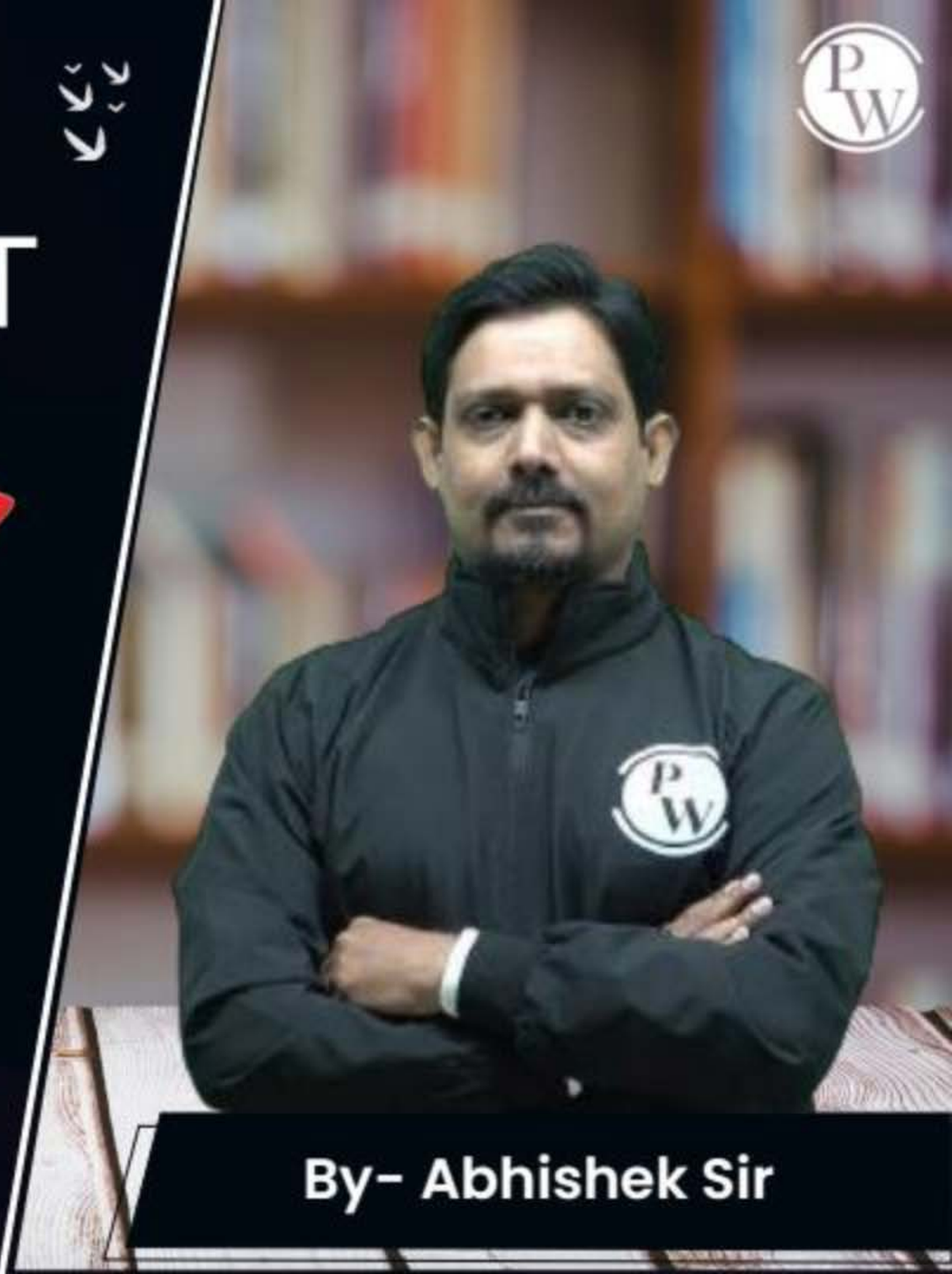
## C Programming

DPD
+
Weekly Test
+
PYQ



## String

Lecture No. 02



By- Abhishek Sir

# Recap of Previous Lecture



- Topic
- Topic
- Topic
- Topic
- Topic

different ways of declaration of string

character array

character pointer

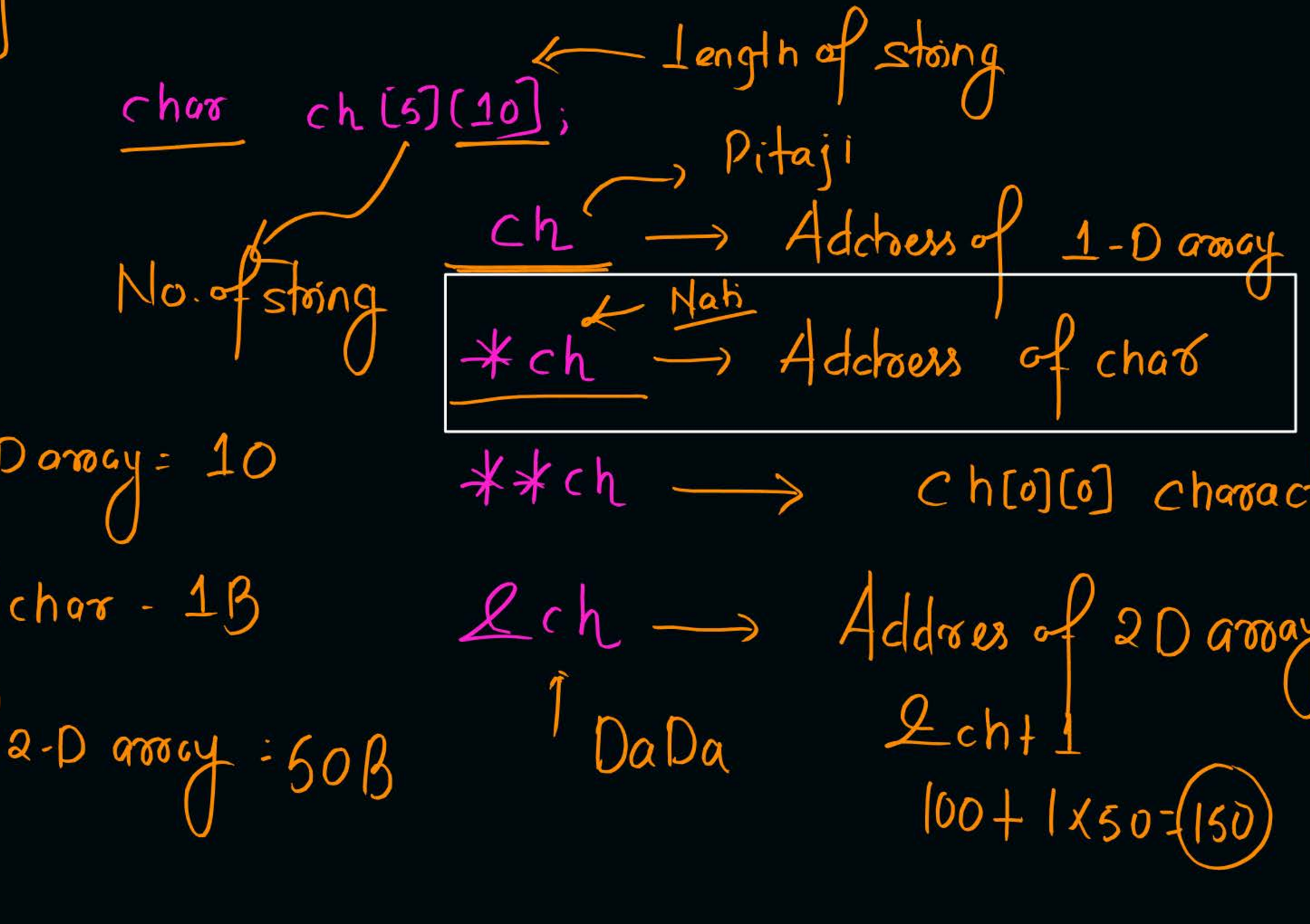
Slide

# Topics to be Covered



- Topic — Array of string
- Topic — using 2D array
- Topic — using array character pointer
- Topic — strcmp(), strcpy(), strcat()
- Topic

# 2-D array


char ch[5][10];
Length of string
No. of string
Pitaji
ch → Address of 1-D array
Nahi
*ch → Address of char
Size of 1-D array = 10
**ch → ch[0][0] character
Size of char - 1B
&ch → Address of 2D array
Size of 2-D array : 50B
DaDa
&ch+1
100+1x50=150

2D

```
printf("%s", *ch);
```

1-D array

Address of character

```
printf("%s", ch);
```



# Agenda

getchar ()
putchar ()



# getchar()

getchar( ) function is used to read one character at a time from the key board. Syntax ch = getchar( ); where ch is a char Var. When this function is executed, the computer will wait for a key to be pressed and assigns the value to the variable when the “enter” key pressed.



# getchar/ putchar

```
int main()
{
        char character;
        character = getchar();

        printf("The entered character is : %c", character);
        return 0;
}
```



# putchar()

The C library **putchar** function is a part of the standard C library and is used to write a single character to the standard output, which is typically the console or terminal.



# getchar/putchar

t.me/Abhisheksharmapw

```
int main()
{
      int character;
      character = getchar();

      printf("The entered character is : %c", character);
      putchar(character);
      return 0;
}
```

Char



# Array of String

```
#include <stdio.h>#include <stdio.h>int
main() {
      char ch[2][10] = {"parakram", "vijay"};
      printf("%c\n",ch[1][3]);
      printf("%s\n",ch[0]);
      printf("%s\n",ch[0]+9);
      printf("%s\n",ch[1]);
      printf("%c\n",ch[2][3]);
return 0;
}
```

0 1

for(i=0; i<=1; i++)
printf("%s\n", ch[i]);

→ a

ch[0] ch[1]

| p | a | r | a | k | r | a | m | \0 | \0 | v 0 | i 1 | j 2 | a 3 | y | \0 | \0 | \0 | \0 | \0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 100 | 101 | 102 | 103 | 104 | 105 | 106 | 107 | 108 | 109 | 110 | 111 | 112 | 113 | 114 | 115 | 116 | 117 | 118 | 119 |

Slide



# Array of String

```
#include <stdio.h>#include <stdio.h>int
main() {
      char ch[2][10] = {"parakram", "vijay"};
      printf("%c\n",ch[1][3]);
      printf("%s\n",ch[0]);
      printf("%s\n",ch[0]+9);
      printf("%s\n",ch[1]);
      printf("%c\n",ch[0][3]);
return 0;
}
```

*(ch+0) = *ch

parakram

| p | a | r | a | k | r | a | m | \0 | \0 | v | i | j | a | y | \0 | \0 | \0 | \0 | \0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 100 | 101 | 102 | 103 | 104 | 105 | 106 | 107 | 108 | 109 | 110 | 111 | 112 | 113 | 114 | 115 | 116 | 117 | 118 | 119 |

Slide

# Array of String

```
#include <stdio.h>#include <stdio.h>int
main() {
      char ch[2][10] = {"parakram", "vijay"};
      printf("%c\n",ch[1][3]);
      printf("%s\n",ch[0]);
      printf("%s\n",ch[0]+9);
      printf("%s\n",ch[1]);
      printf("%c\n",ch[1][3]);
return 0;
}
```

Space ASCII – Nonzero

NULL – ASCII - Zero

1-D

*(ch+0)+9 = *(100+0)

= *(100)

100+9 = 109

↑ character address

Nothing will be printed.

| p | a | r | a | k | r | a | m | \0 | \0 | v | i | j | a | y | \0 | \0 | \0 | \0 | \0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 100 | 101 | 102 | 103 | 104 | 105 | 106 | 107 | 108 | 109 | 110 | 111 | 112 | 113 | 114 | 115 | 116 | 117 | 118 | 119 |



# Array of String

```
#include <stdio.h>#include <stdio.h>int
main() {
      char ch[2][10] = {"parakram", "vijay"};
      printf("%c\n",ch[1][3]);
      printf("%s\n",ch[0]);
      printf("%s\n",ch[0]+9);
      printf("%s\n",ch[1]);
      printf("%c\n",ch[0][3]);
return 0;
}
```

→ vijay

*(ch+1)  *(100+1)

*(110)

110 - Address of ch[1]

| p | a | r | a | k | r | a | m | \0 | \0 | v | i | j | a | y | \0 | \0 | \0 | \0 | \0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 100 | 101 | 102 | 103 | 104 | 105 | 106 | 107 | 108 | 109 | 110 | 111 | 112 | 113 | 114 | 115 | 116 | 117 | 118 | 119 |

Slide



# Array of String

```
#include <stdio.h>#include <stdio.h>int
main() {
      char ch[2][10] = {"parakram", "vijay"};
      printf("%c\n",ch[1][3]);
      printf("%s\n",ch[0]);
      printf("%s\n",ch[0]+9);
      printf("%s\n",ch[1]);
      printf("%c\n",ch[1][3]);
return 0;
}
```

| p | a | r | a | k | r | a | m | \0 | \0 | v | i | j | a | y | \0 | \0 | \0 | \0 | \0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 100 | 101 | 102 | 103 | 104 | 105 | 106 | 107 | 108 | 109 | 110 | 111 | 112 | 113 | 114 | 115 | 116 | 117 | 118 | 119 |

Slide



# Array of String

```
#include <stdio.h>#include <stdio.h>int
main() {
      char ch[2][10] = {"parakram", "vijay"};
      printf("%c\n",ch[1][3]);
      printf("%s\n",ch[0]);
      printf("%s\n",ch[0]+9);
      printf("%s\n",ch[1]);
      printf("%c\n",ch[0][3]);
return 0;
}
```

ch[0][8] = 'k'

ch[0][9] = 'a'

printf("%s", ch[0]);

parakram ka vijay

| p | a | r | a | k | r | a | m | \0 k | \0 a | v | i | j | a | y | \0 | \0 | \0 | \0 | \0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 100 | 101 | 102 | 103 | 104 | 105 | 106 | 107 | 108 | 109 | 110 | 111 | 112 | 113 | 114 | 115 | 116 | 117 | 118 | 119 |

Slide

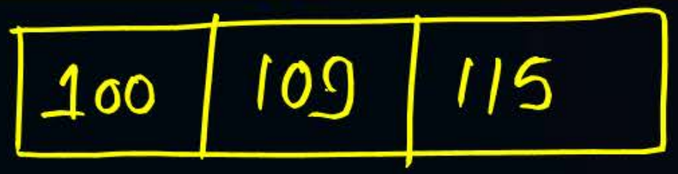


# Array of String

```
include <stdio.h>
int main(){
    char *ch[3] = {"parakram", "vijay","shreshth"};

    printf("%s\n", ch[0]);
    printf("%s\n", ch[1]);
    printf("%s\n", ch[2]);

    return 0;
}
```




*(ch+0)
*(ch+1)
*(ch+2)

When we create array of string using character pointer we need multiple Address hence we declare array of char pointer

ROM

| p | a | r | a | k | r | a | m | \0 | v | i | j | a | y | \0 | s | h | r | e | s | h | t | h | \0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 2 | 2 | 2 | 2 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 0 | 1 | 2 | 3 |



# Array of String

```
#include <stdio.h>
int main() {
    char *ch[2] = {"parakram", "vijay"};
    printf("%s\n",*ch);
    printf("%s\n",*ch+1);
    printf("%s\n",*ch+8);
    printf("%s\n",*(ch+1));
    printf("%c\n",ch[1][3]);
    return 0;
}
```




*(200) = 100

parakram

| p | a | r | a | k | r | a | m | \0 | v | i | j | a | y | \0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 100 | 101 | 102 | 103 | 104 | 105 | 106 | 107 | 108 | 109 | 110 | 111 | 112 | 113 | 114 |

Slide

# Array of String

```
#include <stdio.h>
int main() {
    char *ch[2] = {"parakram", "vijay"};
    printf("%s\n",*ch);
    printf("%s\n",*ch+1);
    printf("%s\n",*ch+8);
    printf("%s\n",*(ch+1));
    printf("%c\n",ch[1][3]);
    return 0;
}
```




*ch

100 + 1 = 101

arakram

| p | a | r | a | k | r | a | m | \0 | v | i | j | a | y | \0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 100 | 101 | 102 | 103 | 104 | 105 | 106 | 107 | 108 | 109 | 110 | 111 | 112 | 113 | 114 |

# Array of String



```
#include <stdio.h>
int main() {
    char *ch[2] = {"parakram", "vijay"};
    printf("%s\n",*ch);
    printf("%s\n",*ch+1);
    printf("%s\n",*ch+8);
    printf("%s\n",*(ch+1));
    printf("%c\n",ch[1][3]);
    return 0;
}
```




*ch + 8
100 + 8 = 108

Nothing

| p | a | r | a | k | r | a | m | \0 | v | i | j | a | y | \0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 100 | 101 | 102 | 103 | 104 | 105 | 106 | 107 | 108 | 109 | 110 | 111 | 112 | 113 | 114 |

Slide

# Array of String

```
#include <stdio.h>
int main() {
    char *ch[2] = {"parakram", "vijay"};
    printf("%s\n",*ch);
    printf("%s\n",*ch+1);
    printf("%s\n",*ch+8);
    printf("%s\n",*(ch+1));
    printf("%c\n",ch[1][3]);
    return 0;
}
```



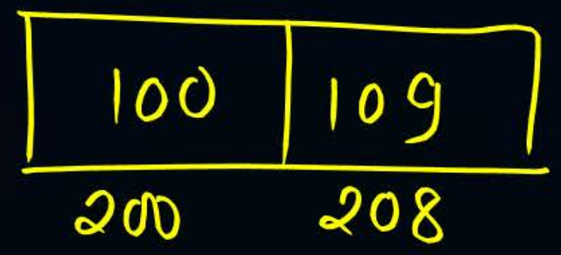

*(ch+1) = ch[1]

→ vijay

| p | a | r | a | k | r | a | m | \0 | v | i | j | a | y | \0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 100 | 101 | 102 | 103 | 104 | 105 | 106 | 107 | 108 | 109 | 110 | 111 | 112 | 113 | 114 |

# Array of String

```
#include <stdio.h>
int main() {
    char *ch[2] = {"parakram", "vijay"};
    printf("%s\n",*ch);
    printf("%s\n",*ch+1);
    printf("%s\n",*ch+8);
    printf("%s\n",*(ch+1));
    printf("%c\n",ch[1][3]);
    return 0;
}
```




$*(*(ch+1)+3)$

$*(*(200+1)+3)$

$*(*(208)+3) = *(109+3)$

| p | a | r | a | k | r | a | m | \0 | v | i | j | a | y | \0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 100 | 101 | 102 | 103 | 104 | 105 | 106 | 107 | 108 | 109 | 110 | 111 | 112 | 113 | 114 |

$*(112) = a$

Slide



# Strcpy

```
strcpy( char *destination, const char *ptr)
```

Const char pointer




```
char str[] = "vijay";
printf("%d", (int) strlen(str));
```

Strlen(100)

const char *ptr)

ptr = 100

`const char *ptr` using Address

stored in ptr we can't

change value of characters



# Strcpy

```
strcpy( char *destination, const char *ptr)
```

```
Char a[] = "parakram";
fun(a);
```

```
void fun( const char *ptr)
{
    ptr[1] = 'i';
}
```



# Strcpy

```
strcpy( char *destination, const char *ptr)
```

strcpy(ch, "parakram");

char ch[10];

ch = "parakram";
X

How to assign

1. one by one using loop
2. strcpy



# Strcpy

Copy the character from source to destination

```
char a[100];

strcpy(a, "hello World");

printf("%s", a);
```

Return type void

Hello world



# Strcpy

```
#include <stdio.h>
#include<string.h>
int main(){
    char ch[20] ="parakram";
    strcpy(ch, "vijay");
    printf("%s", ch);
    return 0;
}
```

vijay

0th Address

| v | i | j | a | y | \0 | a | m | \0 |
|---|---|---|---|---|---|---|---|---|

copy the \0 character also

output : vijay

Slide



# strcmp

String comparison



Slide

# strcmp

Prototype: `int strcmp(const char* s1, const char* s2);`

- Compares two strings s1 and s2
- Returns value
  - Less than 0, if s1 < s2
  - Greater than 0, if s1 > s2
  - Equal to 0, if s1 == s2

$s_1 < s_2$

abc , abd

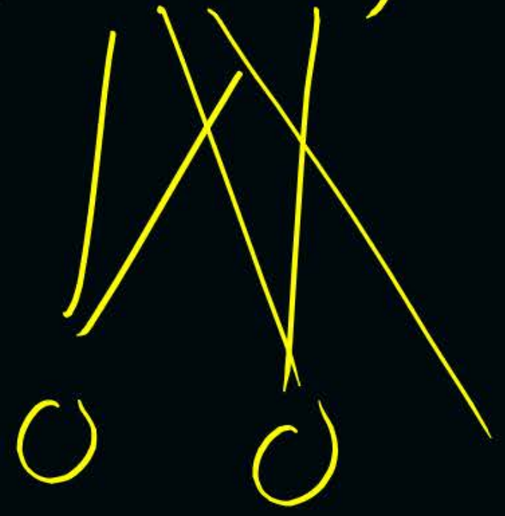
bad abc

98 - 97 = 1

abc abc

ASCII value of each character will be compared

Strcmp(ab, abd);

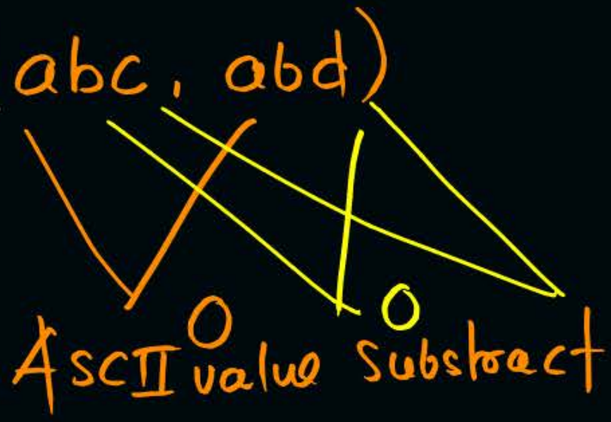


NULL - 100

= -100

$S_1 < S_2$

Strcmp(abc, abd)



ASCII value Substract

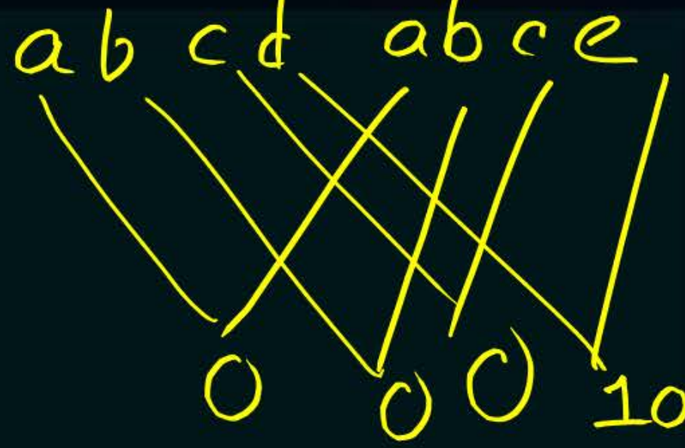
Substract ASCII value of c & d



# strcmp

```c
#include <stdio.h>
#include <string.h>

int main() {
     char *s1 = "abcd";
     char *s2 = "abce";
     if(strcmp(s1, s2) < 0)
     printf("s1 is less than s2");
     else
     printf("s1 is greater than or equal to s2");
    return 0;
```




integer return

$S_1$ less than $S_2$

Slide

# Question

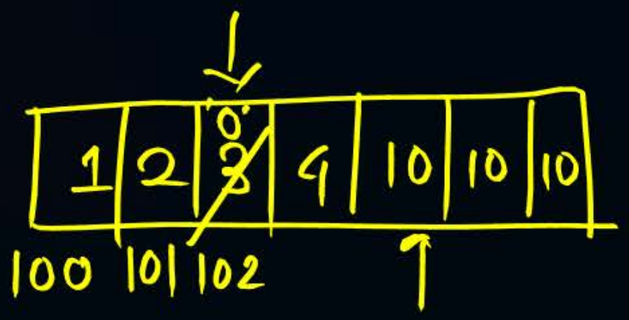
Consider the following C program segment.

```
# include <stdio.h>
int main(){
    char s1[7] = "1234", *p;
    p = s1 + 2;
    *p = '0';
    printf("%s", s1);
}
```

What will be printed by the program?

(A) 12
(B) 120400
(C) 1204
(D) 1034

[C]




*p = s1+2

p | 102

'0' '\0'

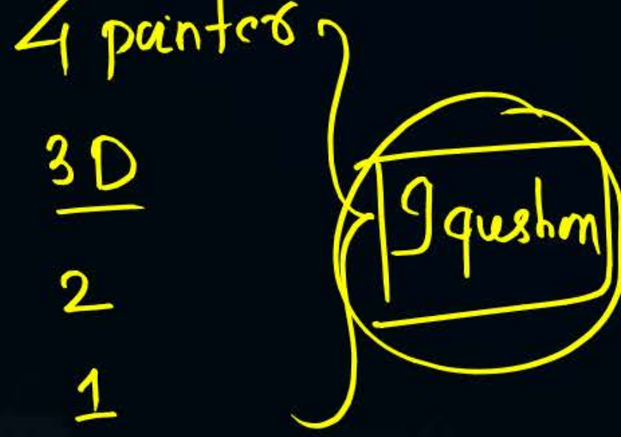
Slide



# Question

What is the output of the program ?

```
#include<stdio.h>
int main(){
   char s[]={'a','b','c','e','c','\0'};
   char *p,*str,*str1;
   p=&s[3];
   str=p;
   str1=s;
   printf("%d",++*p + ++*str1-32);
   return 0;
}
```

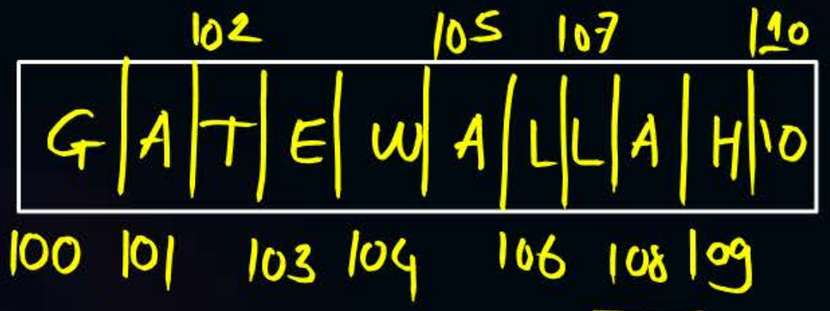



Slide

# GATE 2011

# Q What does the following fragment of C-program print?

```
#include <stdio.h>
#include<string.h>
int main(){
    char c[]="GATEWALLAH";
     char *p =c;
     printf ("%s", p+ p [8] - p [1]);
}
```




(A) Same string

(B) Different string

(C) TE WALLAH

(D) WALLAH

(A)

p 100

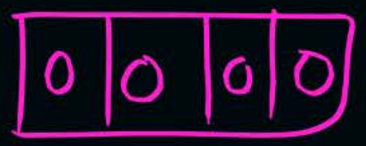
100 + 'A' - 'A'

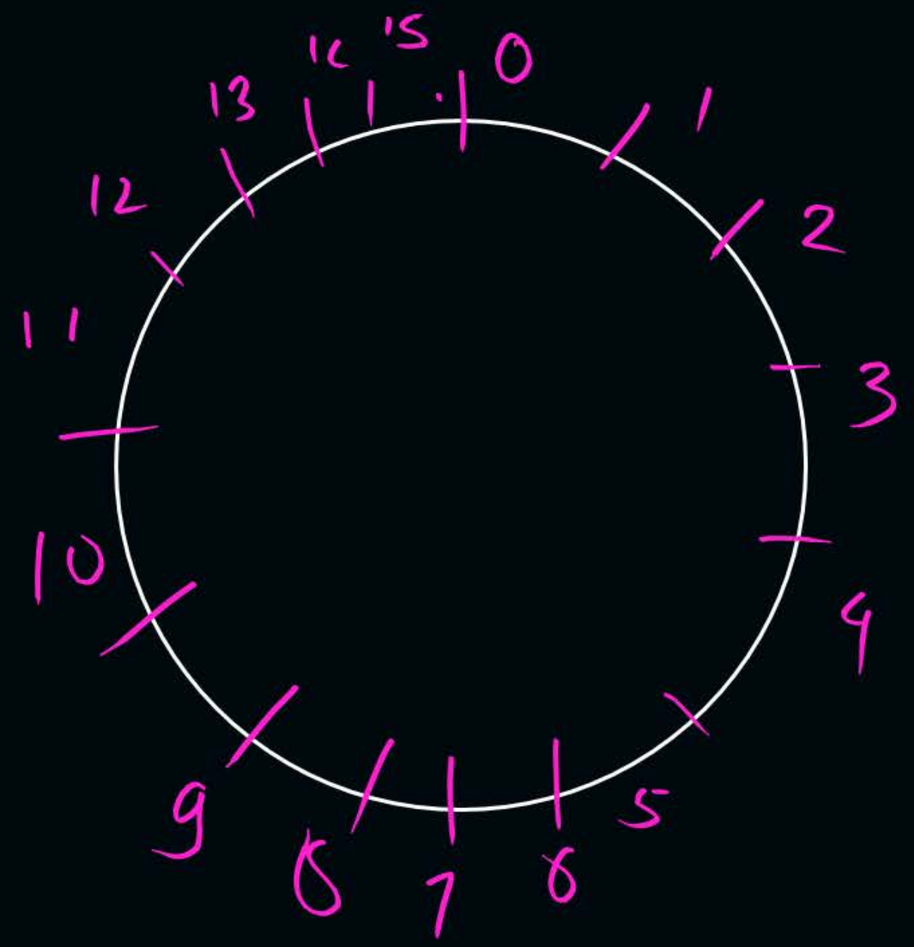
65 - 65

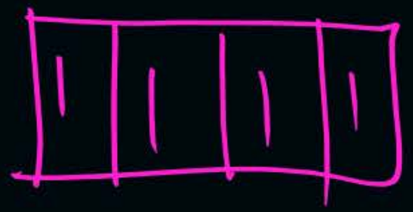
100

Slide

unsing




$0 - 2^n - 1$

only positive side

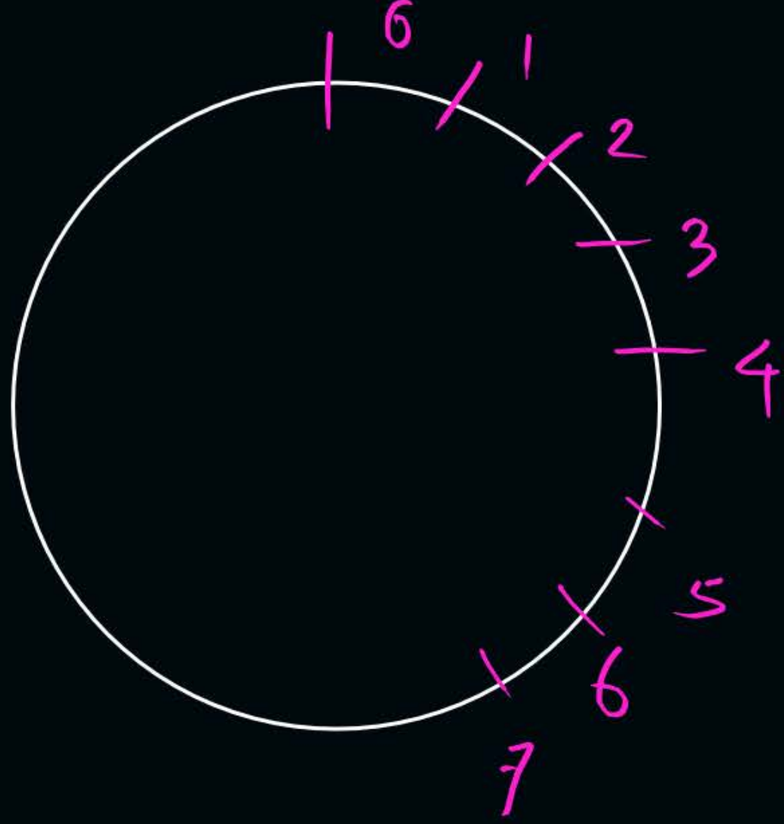
return x++

post

first use old value

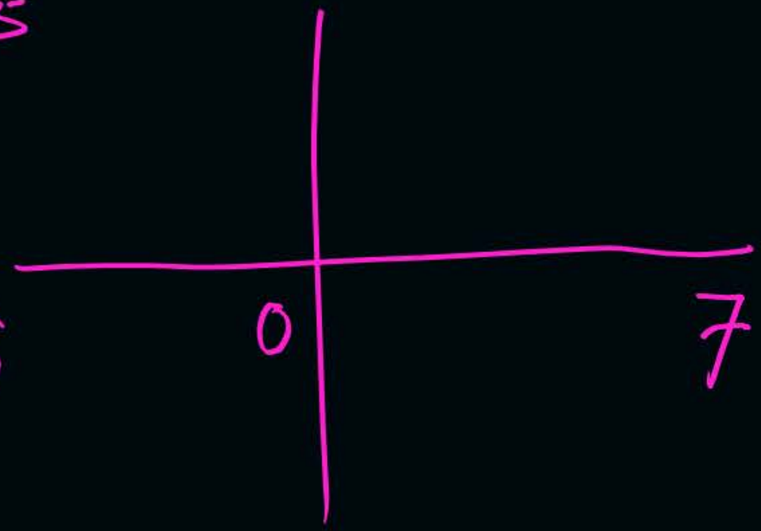
then increment

$-2^{n-1}$ — $2^{n-1}-1$




0 0 0 0

0 1 1 1





# 2 mins Summary

Topic — 2 D array

Topic — Strcmp

Topic — strcpy

Topic

Topic

Slide

# THANK - YOU

